

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

Факультет информационных систем и технологий
Кафедра «Измерительно-вычислительные комплексы»

**Отчет по лабораторной работе №1
по дисциплине «Системы искусственного интеллекта»**

Выполнил:
студент группы ИСТбд-31 Коняхин Н.С

Проверил:
Преподаватель Шишкин В.В.

Ульяновск, 2026

1. Введение и цель

В рамках лабораторной работы была разработана игровая среда «Pacboy» (упрощённый аналог PACMAN) с использованием библиотек Gymnasium и Pygame. Целью работы являлось создание окружения для обучения агента с подкреплением и последующая реализация бота на основе методов глубокого обучения, способного собирать «точки» на карте, избегая столкновений с призраками.

2. Используемая среда для бота и параметры наблюдения

Для взаимодействия с агентом среда реализована как класс `PacboyEnv`, наследующий `gym.Env`. Пространство действий дискретно и содержит 4 возможных хода: вверх, вниз, вправо, влево. Пространство наблюдений задано как `MultiDiscrete([25, 25, 25, 25, 1, 2])` – это соответствует текущим позициям Pacboy (индекс клетки), двух призраков, количеству оставшихся «точек», награде за последний шаг и флагу окончания игры. Такое представление позволяет агенту получать всю необходимую информацию о состоянии игрового поля.

3. Выбор алгоритма обучения

Для обучения бота предполагается использовать алгоритм **Deep Q-Network (DQN)**, так как он хорошо зарекомендовал себя в задачах с дискретным пространством действий и большим количеством состояний. DQN использует аппроксимацию Q-функции с помощью нейронной сети, что позволяет агенту обучаться на основе собственного опыта, применяя метод воспроизведения опыта (experience replay) и целевую сеть для стабилизации обучения. Этот алгоритм оптимально подходит для игровой среды Pacman, где агент должен учиться максимизировать награду, собирая точки и избегая опасностей.

4. Заключение (вывод)

В ходе лабораторной работы была успешно разработана пользовательская среда PACMAN на базе Gymnasium, включающая отрисовку, механику перемещения и учёт столкновений со стенами. Подготовлена основа для внедрения агента с глубоким обучением, выбран подходящий алгоритм DQN. Полученные результаты могут быть использованы для дальнейшего исследования алгоритмов обучения с подкреплением в игровых задачах.

КОД

```
import gymnasium as gym
from gymnasium import spaces
import pygame as py
import numpy as np

class DotsSprite(py.sprite.Sprite):
    def __init__(self, coord_x, coord_y, cell_index, radius=5, color=(0, 255, 0)):
        super().__init__()

        self.image = py.Surface((radius*2, radius*2), py.SRCALPHA)
        self.cell = cell_index

    py.draw.circle(self.image, color=color, center=(radius, radius), radius=radius)
    self.rect = self.image.get_rect(center=(coord_x, coord_y))
    def delete(self):
        self.kill()

    def get_cell_index(self):
        return self.cell

class PacboyEnv(gym.Env):
    def __init__(self, grid_size = 10, width=800, height=600):
        super().__init__()

        self.grid_size = grid_size

        self.action_space = spaces.Discrete(4)
        self.observation_space = spaces.MultiDiscrete([25, 25, 25, 25, 1, 2])

        self.pacboy_pos = None
        self.ghost1_pos = None
        self.ghost2_pos = None

        self.score = 0
        self.tasty_dots = None
        self.reward = 0
        self.game_flag = True
        py.init()
        self.screen = py.display.set_mode((width, height))
        self.clock = py.time.Clock()
        self.selfsize_wall = 50
        self.list_rect = []
        self.list_dots = []
        self.sprite_dots = py.sprite.Group()
        # py.circle(self.screen, (255, 255, 0), center=coords, radius=15)
        self.background = py.Surface((width, height))
        self.game_map = [
            '#####',
            '#-----#',
            '#-----#',
            '#-----#',
            '#-----#',
            '#-----#',
            '#-----#',
            '#-----#',
            '#-----#',
            '#-----#',
            '#-----#',
            '#####'
        ]

    def reset(self, seed=None, options=None):
```

```

super().reset(seed=seed)

self.pacboy_pos = 15
self.ghost2_pos = 24
self.ghost1_pos = 19
self.sprite_dots = py.sprite.Group()
self.score = 0
self.tasty_dots = self.grid_size * self.grid_size
self.reward = 0
self.game_flag = True
self.list_rect = []
self.list_dots = []

self.map_init(self.game_map)

def get_obs(self):
    return np.array([self.pacboy_pos, self.ghost1_pos, self.ghost2_pos,
                    self.tasty_dots, self.reward, int(self.game_flag)],
dtype=np.int32)
def map_init(self, map):
    coord_y = 0
    coord_x = 0
    cell_index = 0

    for line in map:

        for cell in line:

            if cell == '#':
                self.list_rect.append(cell_index)
                rect_coord = coord_x * self.selfsize_wall, coord_y *
self.selfsize_wall, self.selfsize_wall, self.selfsize_wall
                py.draw.rect(self.background, (255,255,255), rect =
rect_coord )
            if cell == '-':
                self.list_dots.append(cell_index)
                dot = DotsSprite(coord_x=(coord_x * 50)+15, coord_y=(coord_y
* 50)+15, cell_index=cell_index)
                self.sprite_dots.add(dot)
                cell_index += 1
                coord_x += 1
            coord_y += 1
            coord_x = 0

def step(self, action):
    reward = 0
    old_pacboy_pos = self.pacboy_pos
    if action == 0: #вверх
        self.pacboy_pos += self.grid_size
    if action == 1: # вниз
        self.pacboy_pos -= self.grid_size
    if action == 2: # вправо
        self.pacboy_pos += 1
    if action == 3: # влево
        self.pacboy_pos -= 1

    if self.pacboy_pos in self.list_rect:
        self.pacboy_pos = old_pacboy_pos

    if self.pacboy_pos in self.list_dots:
        self.list_dots.remove(self.pacboy_pos)

    reward += 1

```

```

        for dot in self.sprite_dots:
            if dot.get_cell_index() == self.pacboy_pos:
                dot.delete()
                self.tasty_dots -= 1
                break

    terminated = False # условие столкновения

    info = {}

    return
(self.pacboy_pos, self.ghost1_pos, self.ghost2_pos), self.tasty_dots, reward,
terminated, info
    def render(self):
        for event in py.event.get():
            if event.type == py.QUIT:
                self.close()
        self.screen.fill((0, 0, 0)) # чёрный фон

        coords = divmod(self.pacboy_pos, self.grid_size)
        coords = ((coords[1] * 50) + 15, (coords[0] * 50) + 15)

        self.screen.blit(self.background, (0, 0))

        py.draw.circle(self.screen, (255, 255, 0), center=coords, radius=15)
        self.sprite_dots.update()
        self.sprite_dots.draw(self.screen)
        py.display.update()
        self.clock.tick(3) # 30 FPS

    def close(self):
        py.quit()

# Регистрация среды (если нужно использовать gym.make)
from gymnasium.envs.registration import register
register(id='Kolobok-v1', entry_point='__main__:PacboyEnv')

env = gym.make('Kolobok-v1')

env.reset(seed=12545)
while True:
    env.render()
    action = env.action_space.sample()
    print(action)
    obs = env.step(action)
    print(obs)

```